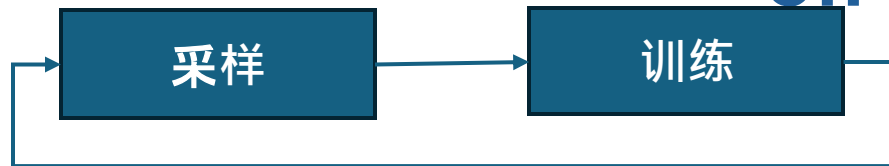


Group Relative Policy Optimization

Mason

2025/11/15

off-policy vs on-policy

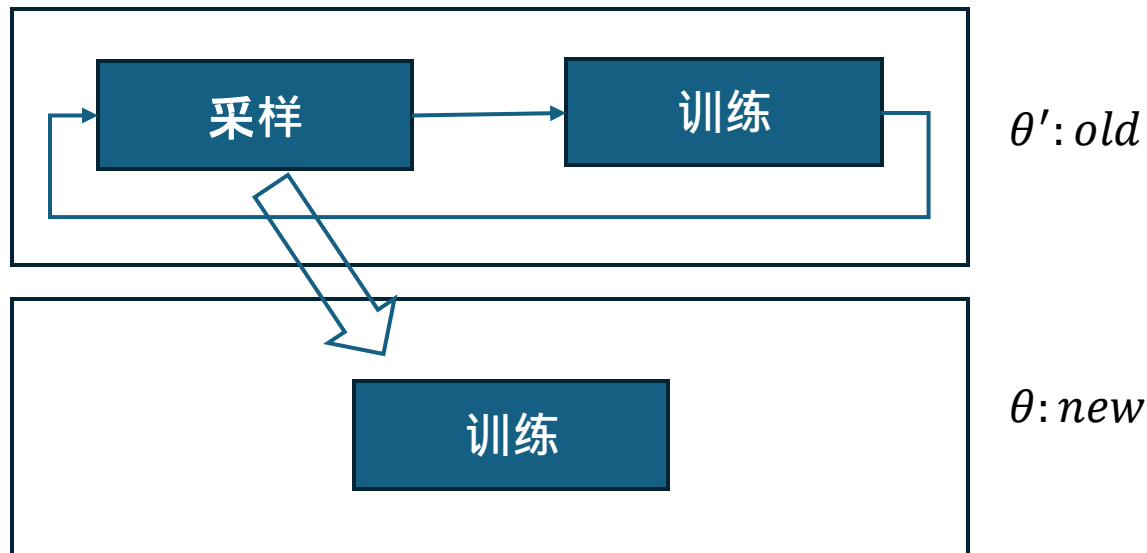


重要性采样: IS Important sampling

Importance weight: $\frac{p_{\theta}(\tau)}{p_{\theta'}(\tau)}$

On-policy:

- **same env**
- 边实践边学习: 数据利用率低
- $E_{\tau \sim p(\theta)}[R(\tau) \nabla \log p_{\theta}(\tau)]$



Off-policy:

- **different env**
- 观察别人学习: 数据利用率高
- $E_{\tau \sim p(\theta)}[\frac{p_{\theta}(\tau)}{p_{\theta'}(\tau)} R(\tau) \nabla \log p_{\theta}(\tau)]$

Preview Lesson

GAE优势函数 Advantage Function

$$A_{\theta}^{GAE}(s_t, a) = \sum_{b=0}^{\infty} (\lambda \gamma)^b \delta_{t+b}^v$$

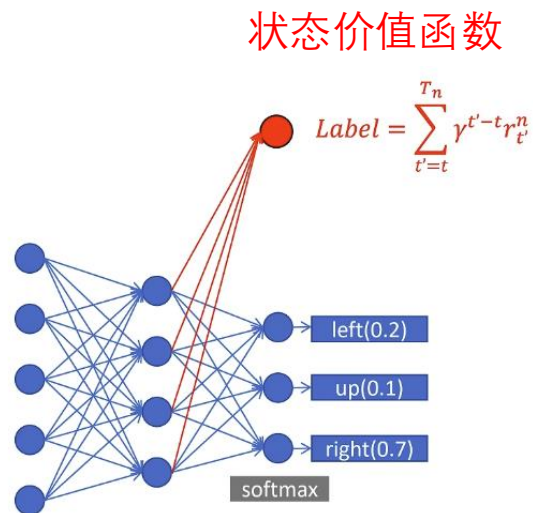
TD-Error 时间差分误差 Temporal Difference Error

$$\delta_t^v = r_t + \gamma * V_{\theta}(s_{t+1}) - \boxed{V_{\theta}(s_t)}$$

梯度策略(Policy Gradient)算法更新规则

$$\frac{1}{N} \sum_{n=1}^N \sum_{t=1}^{T_n} A_{\theta}(s_n^t, a_n^t) \nabla \log P_{\theta}(a_n^t | s_n^t)$$

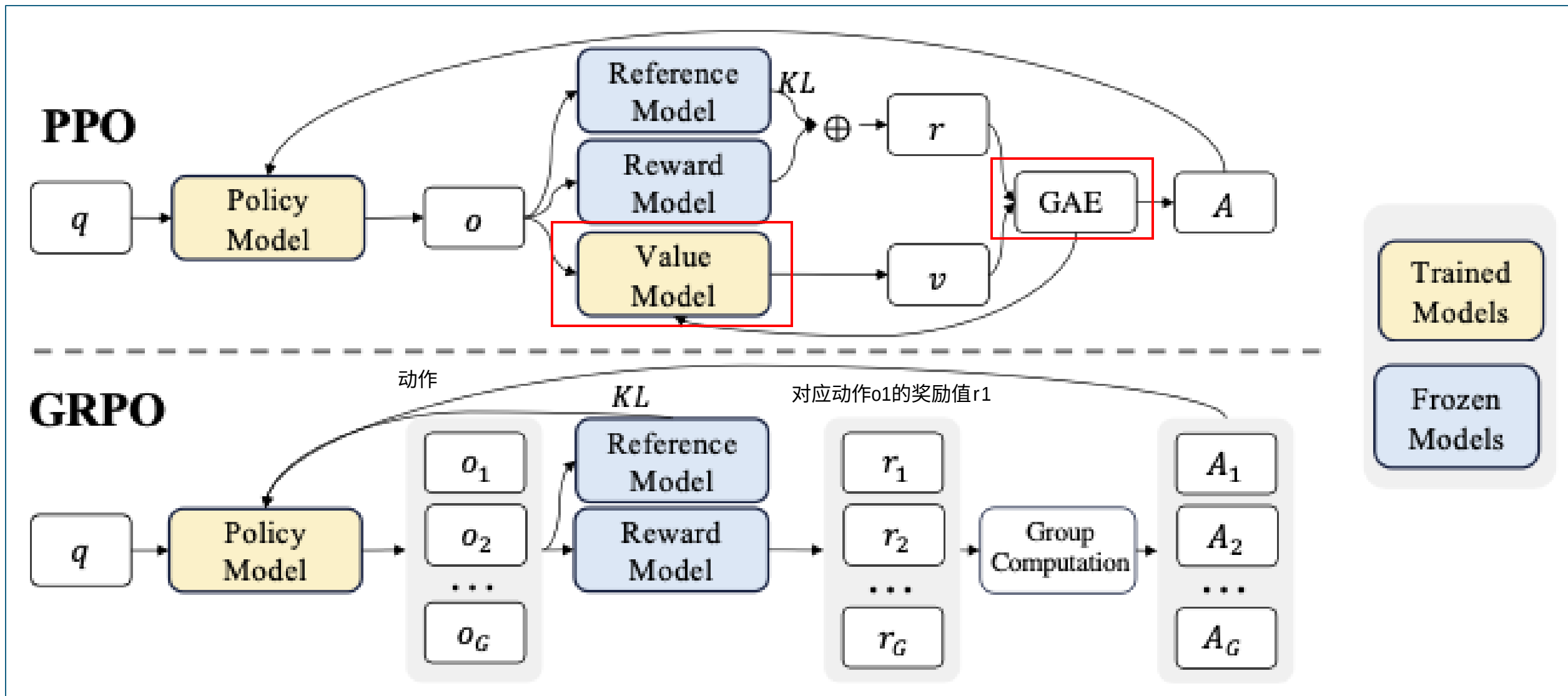
$$\underbrace{A_{\theta}^{GAE}(s_n^t, a_n^t)}_{\text{这步动作有多好? (幅度)}} \times \underbrace{\nabla \log P_{\theta}(a_n^t | s_n^t)}_{\text{如何让这步动作更多/少出现? (方向)}}$$



关键点： 这个 **Label** 必须在一局游戏结束后才能计算出来，因为它需要用到未来的所有奖励信息

Road Map: GRPO <= PPO(CLIP) <= TRPO <= PG(Policy Gradient)

Structure Part



Value Model(GAE): $\delta_t^V = r_t + \gamma * V_{\theta}(s_{t+1}) - V_{\theta}(s_t)$ 这部分是需要训练的

Math

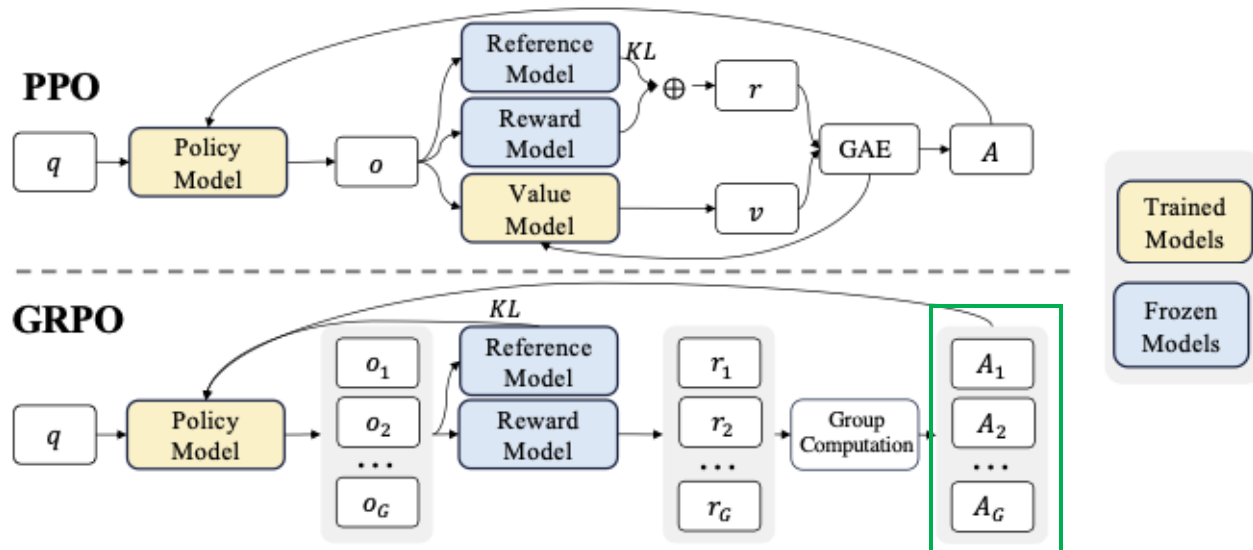


Figure 4 | Demonstration of PPO and our GRPO. GRPO foregoes the value model, instead estimating the baseline from group scores, significantly reducing training resources.

4.1.2. Outcome Supervision RL with GRPO

Formally, for each question q , a group of outputs $\{o_1, o_2, \dots, o_G\}$ are sampled from the old policy model $\pi_{\theta_{old}}$. A reward model is then used to score the outputs, yielding G rewards $\mathbf{r} = \{r_1, r_2, \dots, r_G\}$ correspondingly. Subsequently, these rewards are normalized by subtracting the group average and dividing by the group standard deviation. Outcome supervision provides the normalized reward at the end of each output o_i and sets the advantages $\hat{A}_{i,t}$ of all tokens in the output as the normalized reward, i.e., $\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$, and then optimizes the policy by maximizing the objective defined in equation (3).

$$A_{\theta}(s, a) = Q_{\theta}(s, a) - V_{\theta}(s)$$

$$A_{\theta}(s_t, a) = r_t + \gamma * V_{\theta}(s_{t+1}) - V_{\theta}(s_t)$$

TD-Error

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$$

GRPO创新点

Parsing formula

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL} [\pi_{\theta} || \pi_{ref}] \right\}, \quad (3)$$

Diagram illustrating the parsing of the formula (3):

- $\frac{1}{G} \sum_{i=1}^G$: Group级别 (Group level)
- $\frac{1}{|o_i|} \sum_{t=1}^{|o_i|}$: 求和 (Summation) and 均值 (Mean)
- $\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}$: 重要性采样 (Importance sampling)
- $\hat{A}_{i,t}$: 替代GAE: 整个组内的均值 这里是一个 sentence级别: 对应的tokens
- $\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$: 句子级别 (Sentence level)

一组output: $|o_i|$

As the value function employed in PPO is typically another model of comparable size as the policy model, it brings a substantial memory and computational burden. Additionally, during RL training, the value function is treated as a baseline in the calculation of the advantage for variance reduction. **While in the LLM context, usually only the last token is assigned a reward score by the reward model, which may complicate the training of a value function that is accurate at each token.** To address this, as shown in Figure 4, we propose Group Relative Policy Optimization (GRPO), which obviates the need for additional value function approximation as in PPO, and instead uses the average reward of multiple sampled outputs, produced in response to the same question, as the baseline. More specifically, for each question q , GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy $\pi_{\theta_{old}}$ and then optimizes the policy model by maximizing the following objective:

Token-Level in LLM

传统PPO的问题:

1. 价值函数的作用 $V(s)$: Baseline(基线) 减少方差

$$A_{\theta}(s_t, a) = r_t + \gamma * V_{\theta}(s_{t+1}) - V_{\theta}(s_t)$$

在数学推理任务中:

```
Question: 2+2=?
Token 1: "The"    → reward = 0
Token 2: "answer" → reward = 0
Token 3: "is"     → reward = 0
Token 4: "4"      → reward = 1 ✓ (只有最后才给奖励)
```

关键问题:

- ❌ 只有最后一个token得到奖励分数 (答案对或错)
- ❌ 中间所有tokens的奖励都是0
- ❌ 但价值函数需要为每个token都估计一个准确的价值

想象训练一个价值网络:

```
# 价值函数需要学习:
V("The") = ?           # 这个token值多少?
V("The answer") = ?    # 加上这个token值多少?
V("The answer is") = ? # 再加这个呢?
V("The answer is 4") = 1.0 # 只有这个有明确信号!
```

困难在于:

- 前面的tokens没有直接的奖励信号
- 价值函数要靠时序差分(TD)反向传播学习
- 但只有最后一个token有真实反馈
- 导致价值估计不准确, 方差减少效果差

GRPO的解决方案

核心思想: 用组统计代替学习的价值函数

```
# 对同一个问题, 采样多个答案:
Question: "Solve x^2 - 5x + 6 = 0"

Output 1: "x = 2 or x = 3" → reward = 1.0 ✓
Output 2: "x = 1 or x = 6" → reward = 0.0 ✗
Output 3: "x = 2 or x = 3" → reward = 1.0 ✓
Output 4: "x = 0 or x = 5" → reward = 0.0 ✗

# 组平均作为基线
baseline = mean([1.0, 0.0, 1.0, 0.0]) = 0.5

# 标准化优势
Advantage_1 = (1.0 - 0.5) / std = +0.5 (强化)
Advantage_2 = (0.0 - 0.5) / std = -0.5 (惩罚)
Advantage_3 = (1.0 - 0.5) / std = +0.5 (强化)
Advantage_4 = (0.0 - 0.5) / std = -0.5 (惩罚)
```

SPLIT FUNCTION

Question

Output 1 (o_1)

sentence级别

token 1 ($o_1, 1$)

token 2 ($o_2, 1$)

token t (o_t, t)

Output 2 (o_2)

sentence级别

token 1 ($o_2, 1$)

...

Output G (o_G)

sentence级别

DeepSeekMath-Base 7B with a learning rate of $2e-5$. For GRPO, we set the learning rate of the policy model as $1e-6$. The KL coefficient is 0.04. For each question, we sample 64 outputs. The

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \epsilon, 1 + \epsilon \right) A_t \right], \quad (1)$$

where π_{θ} and $\pi_{\theta_{old}}$ are the current and old policy models, and q, o are questions and outputs sampled from the question dataset and the old policy $\pi_{\theta_{old}}$, respectively. ϵ is a clipping-related

G(Group级别)

- **G** = 组大小, 即对同一个问题采样的输出数量
- 论文设置 **G = 64**
- 最外层的循环

```
Question: "Solve x^2 - 5x + 6 = 0"
```

```
# 采样G=64个完整答案
```

```
Output 1: "First, factor the equation: (x-2)(x-3)=0, so x=2 or x=3"
```

```
Output 2: "Using quadratic formula: x = (5±1)/2, so x=2 or x=3"
```

```
Output 3: "Let me try x=1: 1-5+6=2≠0. Try x=2: 4-10+6=0✓..."
```

```
...
```

```
Output 64: "The solutions are x=1 and x=6" # 错误答案
```

```
# G = 64 (64个不同的完整答案)
```

o_i (Sentence/Output级别)

- o_i = 第*i*个完全输出序列(一个完整答案)'
- 这是一个token序列: $o_i = [o_{i,1}, o_{i,2}, \dots, o_{i,t}]$
- $|o_i|$ = 第*i*个输出的长度(token数量)

```
o1 = [ "First", "(", "x", "-", "2", ")", "(", "x", "-", "3", ")", "=", "0", "so", "x", "=", "2", "or", "x", "=", "3" ]  
# |o1| = 27 tokens
```

```
o2 = [ "Using", "quadratic", "formula", ":", "x", "=", "(", "5", "±", "1", ")", "/", "2", "so", "x", "=", "2", "or", "x", "=", "3" ]  
# |o2| = 22 tokens
```


SPLIT FUNCTION

t (Token级别)

- $t = \text{token}$ 的位置索引
- 范围: 1到 $|o_i|$
- 最细粒度层级

与GAE的对比

$$A_{\theta}^{GAE}(s_t, a) = \sum_{b=0}^{\infty} (\lambda \gamma)^b \delta_{t+b}^v$$

$$\hat{A}_{i,t} = \tilde{r}_i = \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}.$$

优势: 不需要为每个token单独计算优势, 而是用整个输出的奖励

Question

Output 1 (o_1)

sentence级别

token 1 ($o_1, 1$)

token 2 ($o_2, 1$)

token t (o_t, t)

Output 2 (o_2)

sentence级别

token 1 ($o_2, 1$)

...

Output G (o_G)

sentence级别

DeepSeekMath-Base 7B with a learning rate of $2e-5$. For GRPO, we set the learning rate of the policy model as $1e-6$. The KL coefficient is 0.04. For each question, we sample 64 outputs. The

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), o \sim \pi_{\theta_{old}}(O|q)] \frac{1}{|o|} \sum_{t=1}^{|o|} \min \left[\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})} A_t, \text{clip} \left(\frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{\theta_{old}}(o_t|q, o_{<t})}, 1 - \epsilon, 1 + \epsilon \right) A_t \right], \quad (1)$$

where π_{θ} and $\pi_{\theta_{old}}$ are the current and old policy models, and q, o are questions and outputs sampled from the question dataset and the old policy $\pi_{\theta_{old}}$, respectively. ϵ is a clipping-related

Where is KL divergence

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$
$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}] \right\}, \quad (3)$$

where ϵ and β are hyper-parameters, and $\hat{A}_{i,t}$ is the advantage calculated based on relative rewards of the outputs inside each group only, which will be detailed in the following subsections. The group relative way that GRPO leverages to calculate the advantages, aligns well with the comparative nature of rewards models, as reward models are typically trained on datasets of comparisons between outputs on the same question. Also note that, instead of adding KL penalty in the reward, GRPO regularizes by directly adding the KL divergence between the trained policy and the reference policy to the loss, avoiding complicating the calculation of $\hat{A}_{i,t}$.

$$\beta D_{KL}[\pi_{\theta} || \pi_{ref}]$$

```
# 奖励保持简单
r_i = reward_model(o_i) # 只有一个标量

# 优势计算简单
A_i,t = (r_i - mean(r)) / std(r) # 所有tokens一样!

# KL惩罚是整体的
KL_term = D_KL(pi_theta || pi_ref) # 一个标量, 加到总损失上
```

on the rewards $\{r_{\geq t}\}$ and a learned value function V_{ψ} . Thus, in PPO, a value function needs to be trained alongside the policy model and to mitigate over-optimization of the reward model, the standard approach is to add a per-token KL penalty from a reference model in the reward at each token (Ouyang et al., 2022), i.e.,

$$r_t = r_{\phi}(q, o_{\leq t}) - \beta \log \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{ref}(o_t|q, o_{<t})}, \quad (2)$$

where r_{ϕ} is the reward model, π_{ref} is the reference model, which is usually the initial SFT model, and β is the coefficient of the KL penalty.

每个token都有KL惩罚

```
# 原始奖励 (只在最后)
r1 = 0 # token 1
r2 = 0 # token 2
r3 = 0 # token 3
r4 = 1 # token 4 (最后一个)

# 加上per-token KL后
r1 = 0 - beta*KL1 = -0.05
r2 = 0 - beta*KL2 = -0.03
r3 = 0 - beta*KL3 = -0.04
r4 = 1 - beta*KL4 = 0.96

# 现在需要用GAE计算优势:
A1 = r1 + gamma*V(s2) - V(s1) # 复杂!
A2 = r2 + gamma*V(s3) - V(s2) # 需要价值函数
A3 = r3 + gamma*V(s4) - V(s3)
A4 = r4 - V(s4)
```

KL divergence Difference

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}] \right\}, \quad (3)$$

where ϵ and β are hyper-parameters, and $\hat{A}_{i,t}$ is the advantage calculated based on relative rewards of the outputs inside each group only, which will be detailed in the following subsections. The group relative way that GRPO leverages to calculate the advantages, aligns well with the comparative nature of rewards models, as reward models are typically trained on datasets of comparisons between outputs on the same question. Also note that, instead of adding KL penalty in the reward, GRPO regularizes by directly adding the KL divergence between the trained policy and the reference policy to the loss, avoiding complicating the calculation of $\hat{A}_{i,t}$.

$$\beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}]$$

And different from the KL penalty term used in (2), we estimate the KL divergence with the following unbiased estimator (Schulman, 2020):

$$\mathbb{D}_{KL}[\pi_{\theta} || \pi_{ref}] = \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1, \quad (4)$$

which is guaranteed to be positive.

on the rewards $\{r_{\geq t}\}$ and a learned value function V_{ψ} . Thus, in PPO, a value function needs to be trained alongside the policy model and to mitigate over-optimization of the reward model, the standard approach is to add a per-token KL penalty from a reference model in the reward at each token (Ouyang et al., 2022), i.e.,

$$r_t = r_{\phi}(q, o_{\leq t}) - \beta \log \frac{\pi_{\theta}(o_t|q, o_{<t})}{\pi_{ref}(o_t|q, o_{<t})}, \quad (2)$$

where r_{ϕ} is the reward model, π_{ref} is the reference model, which is usually the initial SFT model, and β is the coefficient of the KL penalty.

每个token都有KL惩罚

方面	PPO (公式2)	GRPO (公式3+4)
KL计算方式	$\log(\pi_{\theta}/\pi_{ref})$	$r - \log(r) - 1$ (无偏估计器)
KL位置	加在每个token的奖励中	加在整体损失中
是否保证非负	✗ 可能为负	✓ 保证非负
对优势计算的影响	✗ 使奖励变复杂	✓ 奖励保持简单

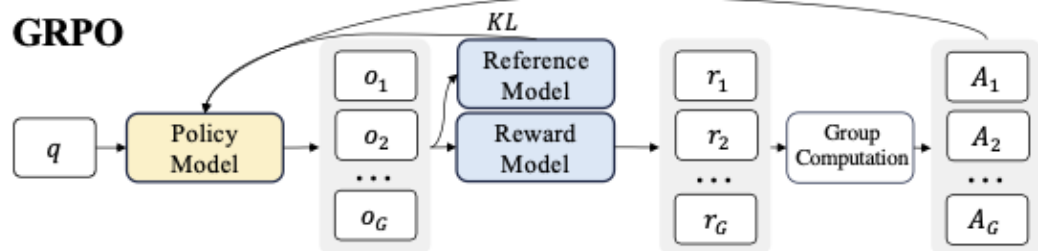
Iterative Group Relative Policy Optimization

Algorithm 1 Iterative Group Relative Policy Optimization

Input initial policy model $\pi_{\theta_{\text{init}}}$; reward models r_{φ} ; task prompts $\mathcal{D}$; hyperparameters ε, β, μ

- 1: policy model $\pi_{\theta} \leftarrow \pi_{\theta_{\text{init}}}$
- 2: **for** iteration = 1, ..., I **do** 迭代次数
- 3: reference model $\pi_{\text{ref}} \leftarrow \pi_{\theta}$ 不希望新的model会偏移基座太多
- 4: **for** step = 1, ..., M **do** 采样数据
- 5: Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
- 6: Update the old policy model $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ old
- 7: Sample G outputs $\{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$ for each question $q \in \mathcal{D}_b$ 采样一组
- 8: Compute rewards $\{r_i\}_{i=1}^G$ for each sampled output o_i by running r_{φ} 运行奖励模型获取r
- 9: Compute $\hat{A}_{i,t}$ for the t -th token of o_i through group relative advantage estimation. 计算优势函数
- 10: **for** GRPO iteration = 1, ..., μ **do**
- 11: Update the policy model π_{θ} by maximizing the GRPO objective (Equation 21)
- 12: Update r_{φ} through continuous training using a replay mechanism.

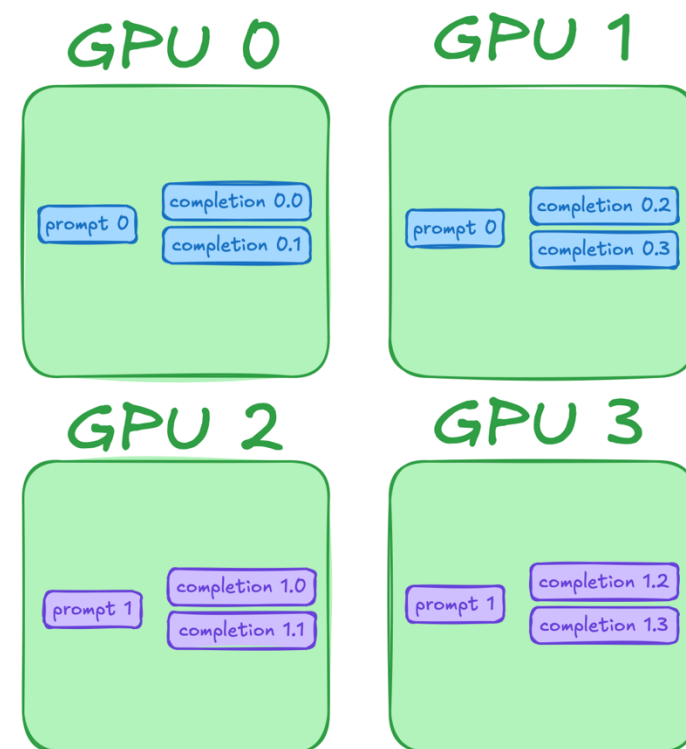
Output π_{θ}



Code Part

```
output_dir="outputs/Qwen-0.5B-GRPO"
run_name="Qwen-1.5B-GRPO-gsm8k"
training_args = GRPOConfig(
    output_dir=output_dir,
    run_name=run_name,
    learning_rate=5e-6,
    adam_beta1 = 0.9,
    adam_beta2 = 0.99,
    weight_decay = 0.1,
    warmup_ratio = 0.1,
    lr_scheduler_type='cosine',
    logging_steps=1,
    bf16=True,
    per_device_train_batch_size=4, # 每个设备上的generation总数
    gradient_accumulation_steps=4,
    num_generations=4, # 表示对每个训练样本（prompt）生成多少个不同的完成序列（completions）这些生成的序列构成一个"组"（group）
    max_prompt_length=256,
    max_completion_length=786,
    num_train_epochs=1,
    save_steps=100,
    max_grad_norm=0.1,
    report_to="wandb",
    log_on_each_node=False,
)
```

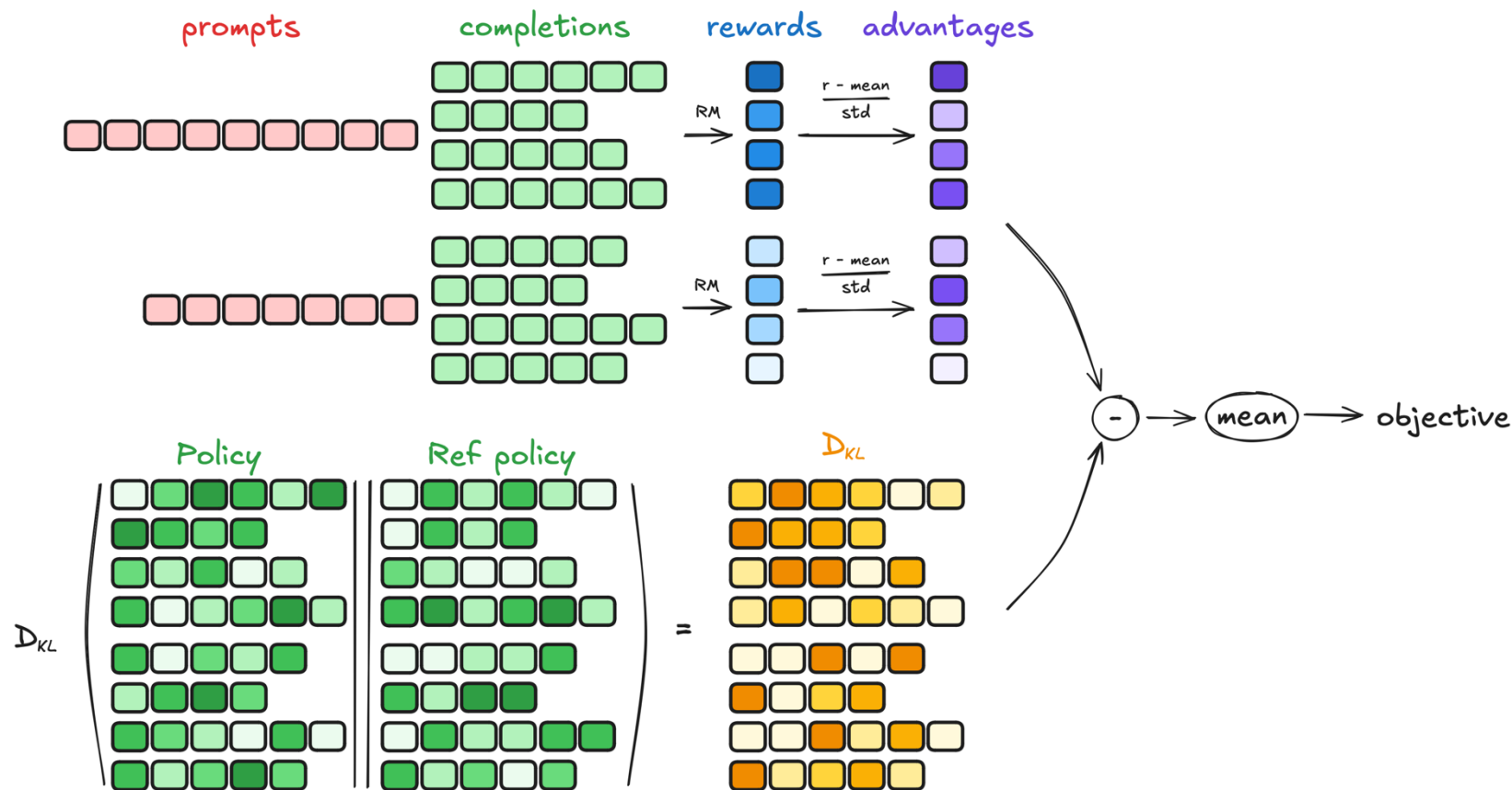
实际prompt数 = $(\text{num_processes} \times \text{per_device_train_batch_size}) / \text{num_generations}$



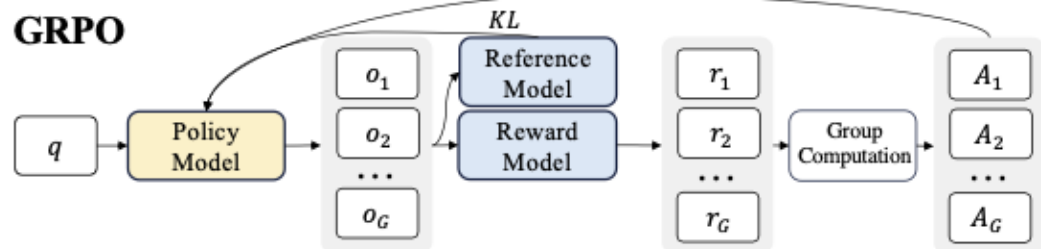
$\text{per_device_batch_size} = 2$
 $\text{num_generations} = 4$

https://huggingface.co/docs/trl/main/en/grpo_trainer

Code Part – Num Generations



GRPO



$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)]$$

$$\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right] - \beta \mathcal{D}_{KL}[\pi_{\theta} || \pi_{ref}] \right\}, \quad (3)$$

Code Part – Arguments On policy

Computing the loss

The objective is to maximize the advantage while ensuring that the model remains close to the reference policy. Consequently, the loss is defined as follows:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \left[\frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{[\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})]_{\text{no grad}}} \hat{A}_{i,t} - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] \right],$$

where the first term represents the scaled advantage and the second term penalizes deviations from the reference policy through KL divergence.

Note that compared to the original formulation in [DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models](#), we don't scale by $\frac{1}{|o_i|}$ because it was shown in the paper [Understanding R1-Zero-Like Training: A Critical Perspective](#) that this introduces a response-level length bias. More details in [loss types](#).

Note that compared to the original formulation in [DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models](#), we use $\beta = 0.0$ by default, meaning that the KL divergence term is not used. This choice is motivated by several recent studies (e.g., [Open-Reasoner-Zero: An Open Source Approach to Scaling Up Reinforcement Learning on the Base Model](#)) which have shown that the KL divergence term is not essential for training with GRPO. As a result, it has become common practice to exclude it (e.g. [Understanding R1-Zero-Like Training: A Critical Perspective](#), [DAPO: An Open-Source LLM Reinforcement Learning System at Scale](#)). If you wish to include the KL divergence term, you can set beta in [GRPOConfig](#) to a non-zero value.

In the original paper, this formulation is generalized to account for multiple updates after each generation (denoted μ , can be set with num_iterations in [GRPOConfig](#)) by leveraging the **clipped surrogate objective**:

$$\mathcal{L}_{\text{GRPO}}(\theta) = -\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \left[\min \left(\frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} \right) - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta} \parallel \pi_{\text{ref}}] \right],$$

where $\text{clip}(\cdot, 1 - \epsilon, 1 + \epsilon)$ ensures that updates do not deviate excessively from the reference policy by bounding the policy ratio between $1 - \epsilon$ and $1 + \epsilon$. When $\mu = 1$ (default in TRL), the clipped surrogate objective simplifies to the original objective.

在计算loss时，除以序列长度 $|o_i|$ ：

$$\text{Loss}_i = -\frac{1}{|o_i|} \sum_t \hat{A}_i \cdot \log \pi_{\theta}(a_t | s_t)$$

动机：让长短不同的completions在同一scale上比较

TRL库的修改：

不除以长度， 直接计算：

$$\text{Loss}_i = -\sum_t \hat{A}_i \cdot \log \pi_{\theta}(a_t | s_t)$$

争议核心：**Response-level Length Bias**（响应级长度偏差）

问题分析：

场景：假设同一个prompt生成了两个completions：

- Completion A：10个token， 平均reward = 0.5， 总reward = 5
- Completion B：100个token， 平均reward = 0.5， 总reward = 5

如果除以长度：

```
ini

Loss_A = advantage × log_prob_sum / 10
Loss_B = advantage × log_prob_sum / 100
```

- 结果：短回答的每个token权重更大
- 偏差：模型可能学会生成过短的回答来最大化per-token reward

特性	图1: no_grad	图2: θ_old
分母含义	当前策略（停止梯度）	历史策略（真实快照）
ratio值	恒等于1	随迭代变化
需要clipping	 否	 是
数据使用次数	1次	μ次
适合μ	μ=1	μ≥1
实现复杂度	简单	中等
TRL默认		

Code Part – Prepare Inputs

```
@profiling_decorator
def _prepare_inputs(self, generation_batch: dict[str, torch.Tensor | Any]) -> dict[str, torch.Tensor | Any]:
    # Prepares inputs for model training/evaluation by managing completion generation and batch handling.
    # During training:
    # - Receives the local generation batch (Per-GPU batch size * steps per generation)
    #   from the modified training dataloader instead of the standard local batch
    # - Generates completions once for the entire generation batch and splits it into batches of size
    #   `per_device_train_batch_size`
    # - Buffers these completions and returns the appropriate slice for the current accumulation step
    # - Optimizes by regenerating completions only periodically (every steps_per_generation * num_iterations)
    # During evaluation:
    # - The input is treated as a standard local batch (no accumulation, no multiple iterations)
    # - Completions are generated for each batch without buffering or reuse
    # Returns a single local batch in both cases.

    mode = "train" if self.model.training else "eval"
    if mode == "train":
        generate_every = self.args.steps_per_generation * self.num_iterations
        if self_step % generate_every == 0 or self._buffered_inputs is None:
            # self._buffered_inputs=None can occur when resuming from a checkpoint
            generation_batch = self.generation_batch
            generation_batch = split_pixel_values_by_grid(generation_batch)
            generation_batch = shuffle_sequence_dict(generation_batch)
            generation_batches = split_tensor_dict(generation_batch, self.args.steps_per_generation)
            self._buffered_inputs = [unsplit_pixel_values_by_grid(batch) for batch in generation_batches]
        inputs = self._buffered_inputs[self_step % self.args.steps_per_generation]
        self_step += 1
    else:
        # In evaluation, there is neither batch grouping for generation, nor multiple iterations, hence
        # local generation batch == local eval batch
        inputs = self.generate_and_score_completions(generation_batch)
    return inputs
```

```
# Calculate rewards for each reward function. rewards_per_func aggregates rewards across all processes. This is
# important because rewards will be normalized per group, and completions are distributed. We will later slice
# rewards_per_func to extract each process's subset.
rewards_per_func = self._calculate_rewards(inputs, prompts, completions, completion_ids_list)

# Apply weights to each reward function's output and sum
rewards = (rewards_per_func * self.reward_weights.to(device)).unsqueeze(0).nansum(dim=1)

# Compute grouped-wise rewards
mean_grouped_rewards = rewards.view(-1, self.num_generations).mean(dim=1)

# Normalize the rewards to compute the advantages
mean_grouped_rewards = mean_grouped_rewards.repeat_interleave(self.num_generations, dim=0)
advantages = rewards - mean_grouped_rewards
```

```
output = {
    "prompt_ids": prompt_ids,
    "prompt_mask": prompt_mask,
    "completion_ids": completion_ids,
    "completion_mask": completion_mask,
    "advantages": advantages,
    "num_items_in_batch": num_items_in_batch,
}
```


Code Part – Compute Loss

```
# Compute the KL divergence between the model and the reference model
if self.beta != 0.0:
    ref_per_token_logps = inputs["ref_per_token_logps"]
    per_token_kl = (
        torch.exp(ref_per_token_logps - per_token_logps) - (ref_per_token_logps - per_token_logps) - 1
    )
```

And different from the KL penalty term used in (2), we estimate the KL divergence with the following unbiased estimator (Schulman, 2020):

$$D_{KL}[\pi_{\theta}||\pi_{ref}] = \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - \log \frac{\pi_{ref}(o_{i,t}|q, o_{i,<t})}{\pi_{\theta}(o_{i,t}|q, o_{i,<t})} - 1, \quad (4)$$

which is guaranteed to be positive.

```
# Compute the loss
advantages = inputs["advantages"]
# When num_iterations == 1 and steps_per_generation <= gradient_accumulation_steps,
# old_per_token_logps == per_token_logps. In this case we can skip its computation
# (see _generate_and_score_completions) and instead use per_token_logps.detach().
# The exception is when using vLLM, where we always compute old_per_token_logps
# for importance sampling
old_per_token_logps = inputs.get("old_per_token_logps")
old_per_token_logps = per_token_logps.detach() if old_per_token_logps is None else old_per_token_logps
```

```
coef_1 = torch.exp(log_importance_weights)
coef_2 = torch.clamp(coef_1, 1 - self.epsilon_low, 1 + self.epsilon_high)

# Two-sided clipping
if self.args.delta is not None:
    coef_1 = torch.clamp(coef_1, max=self.args.delta)

per_token_loss1 = coef_1 * advantages.unsqueeze(1)
per_token_loss2 = coef_2 * advantages.unsqueeze(1)
per_token_loss = -torch.min(per_token_loss1, per_token_loss2)
if entropy_mask is not None:
    per_token_loss = per_token_loss * entropy_mask
```

Discussion